AIM

Adaptive Isolating Model

Compression Algorithm

Canonical Specification — Version 16

March 2026

This document is the canonical specification for the AIM compression algorithm, version 16. It is sufficient to implement a correct encoder and decoder in any programming language without reference to any other source. The reference implementation is `aim_v15.py`.

Version 16 adds **HALT_ANS_STRIDE** (halt code 4) — a fourth halt condition that allows the recursive encoder to stop at any depth and encode the current aligned stream with rANS order-1 stride-k when doing so produces a smaller result than continuing to recurse. This formalises the optimal early cutoff mechanism and defines the rANS codec, stride selection algorithm, and wire format to implementable precision.

1. Overview

AIM (Adaptive Isolating Model) is a lossless data compression algorithm. It operates by recursively decomposing a byte stream into bit-plane components, encoding each component with the information-theoretic model best suited to its structure, and reducing the symbol space at each level. The algorithm is fully deterministic, has no parameters, and terminates on a mathematical proof rather than a heuristic.

AIM achieves compression ratios below raw gzip on audio, video, sensor data, and other structured binary formats by exploiting two classes of structure that a byte-level sliding-window compressor cannot reach:

- **Bit-plane density asymmetry** — exploited by the recursive sweep and flag-set compression race.
- **Inter-symbol value correlation at non-unit stride** — exploited by HALT_ANS_STRIDE on the aligned stream before remap dissolves it.

Property	Value
Magic bytes	0x41 0x49 0x4D 0x34 (ASCII: AIM4)
Max symbol depth	8 levels (one per bit of a byte)
Halt conditions	4 (structural, not heuristic)
Flag formats	7 (race selects minimum at each level)
Terminal backends	2 (gzip, rANS order-1 stride-k)
Encode modes	2 (Recursive, CAIM)
Integrity check	SHA-256 over original data
Container overhead	45 bytes fixed
Reference implementation	aim_v15.py (MIT License)

2. Algorithm

AIM consists of five operations that repeat at each level, plus four halting conditions. The encoder runs all five operations at every depth before deciding whether to halt.

2.1 The Five Operations

Operation 1: Entropy Scan (Sweep)

For each bit position `b` in {0–7}, count the number of bytes with bit `b` set. Let `k_b` be this count and `n` the stream length. Select the bit position with minimum `k_b`. This is the sparsest bit plane — furthest from maximum entropy (d = 0.5). Ties broken by lowest bit index.

Operation 2: Separation (Bit-Clear)

Let `b*` be the selected bit. Produce two objects:

- Flag set `F`: sorted list of all byte positions where bit `b*` is set. A subset of {0…n−1} of size `k_{b*}`.
- Aligned stream `A`: the stream with bit `b*` cleared in every byte. `A[i] = stream[i] AND NOT(1 << b*)`

These two objects are lossless: the original stream is exactly recoverable from (`A`, `F`, `b*`) by setting bit `b*` in `A[i]` for every `i` in `F`.

Operation 3: ANS-Stride Candidate Computation

Before remapping, compute `ans_candidate = ans_stride_encode(A)`. This encodes the aligned stream with rANS order-1 stride-k (see Section 5) and stores the result alongside the level. It is only written to the output if the optimal cutoff analysis (Section 2.3) selects this depth for HALT_ANS_STRIDE. If the level is not selected as the cutoff, the candidate is discarded. This operation has no effect on the flag race or the remap step.

Operation 4: Flag Set Compression (Race)

Encode F using all applicable format encoders simultaneously. Select the encoder that produces the smallest output. The seven formats and their selection conditions are defined in Section 4.

Operation 5: Symbol Space Reduction (Remap)

The aligned stream `A` has bit `b*` cleared in every byte. Remap `A` to remove the gap: for each byte value `v` produce `v'` by deleting bit `b*` from the binary representation and shifting higher bits down:

```
high = v >> (b* + 1)            # bits above b*
low  = v & ((1 << b*) - 1)     # bits below b*
v'   = (high << b*) | low      # pack out the gap
```

After remap, byte values are in [0, 2^(7−depth)−1]. The symbol alphabet has strictly halved. The inverse operation (unremap) reinserts a zero bit at position `b*` and shifts higher bits up.

2.2 Halt Conditions

There are exactly four halting conditions. The first three are decided in O(n) time and require no parameters. The fourth is decided by a size comparison computed during bottom-up assembly.

Code	Name	Trigger	Child Payload
0	HALT_RECURSE	Default — not a terminal	Next level header
1	HALT_TERMINAL	MAX_DEPTH reached, or trivial terminal	gzip(aligned stream)
2	HALT_ZERO	Aligned stream is all-zero bytes	None (child_len = 0)
3	HALT_ONE	Aligned stream is all-one bytes (value 1)	None (child_len = 0)
4	HALT_ANS_STRIDE	ANS-stride encoding of aligned stream is smaller than continuing to recurse	ANS-stride payload (self-describing, see Section 5)

HALT_ZERO and HALT_ONE are structural: they mean the bit-plane decomposition has reached a trivially encodable state. HALT_TERMINAL fires when MAX_DEPTH = 8 is reached. HALT_ANS_STRIDE fires when the aligned stream contains inter-symbol value correlations that are more efficiently captured by direct rANS encoding than by further recursive bit-plane separation.

2.3 Optimal Cutoff Selection (HALT_ANS_STRIDE Logic)

After completing the full descent (collecting all levels up to the natural terminal), the encoder performs bottom-up assembly. At each depth, it compares two options:

```
option_recurse     = size of current payload (all remaining levels + terminal)
option_ans_stride  = 11 + len(ans_candidate[depth])
                   # 11-byte level header + ANS payload

if option_ans_stride < option_recurse:
    emit HALT_ANS_STRIDE at this depth
    discard all deeper levels
else:
    emit HALT_RECURSE and continue assembling
```

This comparison is applied at every depth during the single bottom-up pass. Because assembly proceeds from deepest to shallowest, the `option_recurse` value at each depth is exactly the total byte cost of all remaining structure below that point. No lookahead or approximation is required. The decision is globally optimal within the recursive structure: it finds the earliest depth at which stopping saves bytes.

The encoder stores an ANS-stride candidate at *every* depth during the descent. These are byte buffers held in memory and discarded after assembly. For files where HALT_ANS_STRIDE never fires (e.g. random data, already-compressed data), all candidates are discarded and the output is identical to v14.

3. Theoretical Basis for HALT_ANS_STRIDE

3.1 The Remap Dissolution Problem

The recursive remap operation is mathematically correct for bit-plane entropy separation: at each level it halves the symbol alphabet and removes the structural dependency introduced by the cleared bit. However, it also scrambles *inter-symbol value correlations* that are independent of bit-plane structure.

Consider a stream of 16-bit PCM audio samples stored as consecutive bytes (low byte, high byte). After bit-clearing and remapping at depth 0, the low-byte and high-byte of each sample are now at distance 2 in the remapped stream, but the sample-to-sample correlation — that consecutive samples are numerically close — has been partially destroyed by the bit reshuffling. By depth 2–3, the original inter-symbol structure is effectively gone.

The aligned stream at each depth — post-bit-clear, pre-remap — is the last point at which the original value correlation is intact. HALT_ANS_STRIDE intercepts the stream at this point when the rANS encoder can exploit it more efficiently than the remaining recursive structure.

3.2 Stride Correlation as a Non-Domain Measurement

HALT_ANS_STRIDE requires no knowledge of the data type. The stride `k` is selected empirically by measuring $H(X_i \mid X_{i-k})$ — the conditional entropy of the aligned stream at lag `k` — across a set of candidate values:

```
STRIDE_CANDIDATES = [1, 2, 3, 4, 6, 8, 12, 16]

H(X_i | X_{i-k}) = -sum_{x,c} P(X_i=x, X_{i-k}=c) * log2(P(X_i=x | X_{i-k}=c))

k_opt = argmin_k  H(X_i | X_{i-k})

# Activate only if gain over k=1 exceeds threshold:
if H(k=1) - H(k_opt) < STRIDE_GAIN_THRESH (0.05 bits/sym):
    k_opt = 1    # fall back; no meaningful periodicity found
```

The measurement is computed on a 1 MB prefix of the aligned stream for efficiency. The result is data-driven: the same algorithm finds k=4 for YUV420, k=2 for 16-bit PCM, k=4 for RGBA, k=8 for 8-byte struct records, and k=1 (activating global-only tables) for structureless data. No data type is named or detected.

The gain threshold (0.05 bits/sym) prevents false activation on near-random data where measurement noise would otherwise select an arbitrary large `k` with negligible actual gain. In practice, random data achieves ~0.0002 bits/sym gain, which is 250× below the threshold.

3.3 Why Not the Flag Race, and Why Not a Final Pass

Two alternatives were evaluated and rejected:

Adding ANS-stride to the flag race (as format 7 applied to the flag bitset bytes) was implemented and tested. It consistently lost to FMT_LZ77HUFF at every depth and every data type. The diagnosis: the flag bitset encodes *where* flags appear, not *what values* the aligned stream contains. YUV period-4 and PCM period-2 correlations live in the value domain. gzip already captures whatever positional regularity exists in the flag positions. ANS applied to the wrong signal gains nothing.

A final ANS pass after full encoding was also considered. Compressed data presents as near-uniform random bytes by design — this is the defining property of effective entropy coding. A probabilistic model applied to already-compressed bytes finds no exploitable structure and adds overhead. The correct intervention point is the aligned stream *before* remap, not after terminal encoding.

4. Flag Set Compression Race

The encoder evaluates all applicable formats for the current flag density and stores the smallest result. The format ID stored in the level header identifies the winning format to the decoder.

ID	Name	Applied When	Notes
0	FMT_GAP	density ≤ 0.30	Delta-encoded gap list. Skipped at high density to avoid O(k) memory materialisation.
1	FMT_BITSET	always	Raw packed bitset bytes (n/8 bytes). Never wins but always competes as fallback.
2	FMT_EF	density ≤ 0.15	Elias-Fano. Optimal for sparse uniform distributions.
3	FMT_RLE	always	Gamma-coded run lengths. Wins for bursty bit planes regardless of density.
4	FMT_HUFFMAN	always	Order-0 canonical Huffman on bitset bytes. Wins when byte-value entropy dominates.
5	FMT_LZ77	0.15 ≤ d ≤ 0.85	LZ77 on bitset bytes. Wins rarely; included for structural completeness.
6	FMT_LZ77HUFF	always	LZ77 tokens then Huffman (= DEFLATE/gzip on bitset). Wins for spatially-correlated bit planes.

FMT_LZ77HUFF and FMT_HUFFMAN always compete because Huffman is the correct order-0 entropy coder and LZ77+Huffman additionally finds spatial repetition. FMT_RLE always competes because bursty flag patterns (long runs of set or unset bits) produce very compact gamma-coded run lists. FMT_GAP and FMT_EF are skipped above their density thresholds because materialising a position list at high density is structurally more expensive than the bitset representation they would be compressing.

5. rANS Order-1 Stride-k Codec

The rANS codec encodes a byte stream using an order-1 context model with stride-k predecessor. The context for byte at position `i` is `data[i - k]`. For positions where `i < k`, a sentinel context `ctx = 256` is used (this value is outside the byte range 0–255 and never collides with a real per-context table).

5.1 Constants

```
ANS_M_BITS = 14
ANS_M      = 1 << ANS_M_BITS   # 16384  -- normalisation interval size
ANS_L      = ANS_M              # lower renorm bound
ANS_B      = 256               # output alphabet (bytes)
```

5.2 Frequency Table Normalisation

Given raw symbol counts `raw: {symbol -> count}`, produce a frequency table `freq` summing exactly to `ANS_M`:

```
total = sum(raw.values())
freq  = { s: max(1, round(cnt * ANS_M / total))
          for s, cnt in raw.items() if cnt > 0 }

# Adjust to sum exactly ANS_M
delta = ANS_M - sum(freq.values())
keys  = sorted(freq, key=lambda s: -freq[s])   # descending frequency
i = 0
while delta != 0:
    s = keys[i % len(keys)]
    if delta > 0:              freq[s] += 1; delta -= 1
    elif freq[s] > 1:          freq[s] -= 1; delta += 1
    i += 1

# Cumulative table
cum = {}; acc = 0
for s in range(256): cum[s] = acc; acc += freq.get(s, 0)

# Decode slot array
slots = [0] * ANS_M
for s, f in freq.items():
    for j in range(f): slots[cum[s] + j] = s
```

5.3 Per-Context Table Construction

The encoder builds one table per context byte that has at least 32 observations:

```
from collections import Counter, defaultdict

global_raw  = Counter(data)                     # global counts
ctx_raw     = defaultdict(Counter)              # per-context counts

for i in range(k_opt, n):
    ctx_raw[ data[i - k_opt] ][ data[i] ] += 1

global_freq, global_cum, _ = normalise(dict(global_raw))

ctx_tables = {}
for ctx_byte, cnts in ctx_raw.items():
    if sum(cnts.values()) >= 32:
        ctx_tables[ctx_byte] = normalise(dict(cnts))
```

The global table is used as a fallback for positions with sentinel context (`i < k_opt`) and for context bytes that have fewer than 32 observations.

5.4 Encode

Iterate symbols in reverse order. For each symbol, renorm the ANS state by emitting bytes LSB-first until the state fits the renorm bound, then incorporate the symbol:

```
out = bytearray()
x   = ANS_L

for i in range(n - 1, -1, -1):
    s   = data[i]
    ctx = data[i - k_opt] if i >= k_opt else 256   # sentinel for i < k
    t   = ctx_tables.get(ctx)                      # None -> use global
    f_s = t[0].get(s, 1) if t else global_freq.get(s, 1)
    c_s = t[1].get(s, 0) if t else global_cum.get(s, 0)

    limit = (ANS_L * ANS_B // ANS_M) * f_s        # renorm threshold
    while x >= limit:
        out.append(x & 0xFF); x >>= 8
    x = (x // f_s) * ANS_M + c_s + (x % f_s)     # symbol absorption

# Emit final state (4 bytes, big-endian) -- stored BEFORE renorm bytes
payload = struct.pack('>I', x) + bytes(out)
```

5.5 Decode

Read initial ANS state from the first 4 bytes of the payload (big-endian). Reverse the remaining renorm bytes so they are consumed in the same order they were emitted:

```
x,   = struct.unpack_from('>I', payload, 0)
buf  = payload[4:][::-1]    # renorm bytes reversed
rpos = 0

out = bytearray(n)
for i in range(n):
    ctx   = out[i - k_opt] if i >= k_opt else 256
    t     = ctx_tables.get(ctx)
    slots = t[2] if t else global_slots
    freq  = t[0] if t else global_freq
    cum   = t[1] if t else global_cum

    slot   = x % ANS_M
    s      = slots[slot]
    out[i] = s
    x      = freq[s] * (x >> ANS_M_BITS) + slot - cum[s]

    while x < ANS_L and rpos < len(buf):
        x = (x << 8) | buf[rpos]; rpos += 1
```

5.6 Wire Format

The ANS-stride payload is self-describing. It does not rely on any external context.

```
k_opt         (1 B)    stride selected (1, 2, 3, 4, 6, 8, 12, or 16)
n             (4 B BE) original byte count
global_freq   (512 B)  256 x uint16 LE -- global frequency table
                       (stored separately to prevent collision with
ctx_byte=255)
n_ctx         (2 B BE) number of per-context tables
n_ctx x [
  ctx_byte    (1 B)    context byte value (0-255)
  freq        (512 B)  256 x uint16 LE -- per-context frequency table
]
payload_len   (4 B BE)
payload       (N B)    state (4 B BE) + renorm bytes (LSB-first)
```

The global frequency table is stored as a fixed 512-byte block **before** the per-context table list. It must not be stored as a per-context entry with a special marker byte, because `ctx_byte = 255` is a valid byte value with its own per-context table. Conflating the global table with context 255 would cause decode to use the wrong distribution for bytes preceded by 0xFF.

6. Stride Selection Algorithm

6.1 Conditional Entropy Measurement

Given a byte stream and a lag `k`, compute $H(X_i \mid X_{i-k})$:

```
def conditional_entropy(data, k):
    n = len(data)
    if n <= k: return 8.0
    pair_cnt = Counter()
    ctx_cnt  = Counter()
    for i in range(k, n):
        c = data[i - k]; s = data[i]
        pair_cnt[(c, s)] += 1
        ctx_cnt[c]       += 1
    total = n - k
    H = 0.0
    for (c, s), cnt in pair_cnt.items():
        H -= (cnt / total) * log2(cnt / ctx_cnt[c])
    return H
```

For large streams a numpy implementation is preferred (O(n) single pass via bincount on pair keys = c*256 + s). Both produce identical results.

6.2 Candidate Selection

```
STRIDE_CANDIDATES  = [1, 2, 3, 4, 6, 8, 12, 16]
STRIDE_SAMPLE      = 1_000_000   # bytes to probe
STRIDE_GAIN_THRESH = 0.05        # bits/sym gain over k=1 required

probe  = data[:STRIDE_SAMPLE] if len(data) > STRIDE_SAMPLE else data
scores = { k: conditional_entropy(probe, k) for k in STRIDE_CANDIDATES }
k_opt  = min(scores, key=lambda k: scores[k])

if scores[1] - scores[k_opt] < STRIDE_GAIN_THRESH:
    k_opt = 1    # no meaningful periodicity found
```

Data Type	Typical k_opt	Reason
16-bit PCM audio	2	Sample bytes at distance 2 (low/high byte of same sample)
YUV420 video	4	Y/U/V component stride in interleaved format
RGBA image data	4	Pixel stride (R/G/B/A at offsets 0/1/2/3)
8-byte struct records	8	Record field alignment
Random / compressed	1	No measurable periodicity; gain below threshold

7. Wire Format

7.1 Container Header (45 bytes, fixed)

```
Offset  Size  Field
0       4     magic = 0x41 0x49 0x4D 0x34  ("AIM4")
4       1     mode  = 0 (RECURSIVE) | 1 (CAIM)
5       8     orig_n  uint64 big-endian  -- original byte count
13      32    sha256  -- SHA-256 of original uncompressed data
45      ...   payload (mode-specific)
```

7.2 Recursive Payload

The recursive payload is a self-describing depth-first sequence of level records.

```
Per-level record:
  bit        (1 B)    bit position cleared at this level (0-7)
  fmt        (1 B)    flag format used (0-6, see Section 4)
  flag_len   (4 B BE) length of flag_data in bytes
  halt       (1 B)    halt code (0-4, see Section 2.2)
  child_len  (4 B BE) length of child payload in bytes
             --  total fixed header: 11 bytes  --
  flag_data  (flag_len B)   flag set encoded in fmt
  child      (child_len B)  depends on halt:
               halt=0  -> next level record (recursive)
               halt=1  -> gzip(aligned stream)
               halt=2  -> empty (child_len = 0)
               halt=3  -> empty (child_len = 0)
               halt=4  -> ANS-stride payload (Section 5.6)
```

The `child_len` field is the total byte length of everything following the fixed 11-byte header for this level, including `flag_data`. Decoders parse flag_data first (`flag_len` bytes), then the child.

7.3 CAIM Payload

```
n_levels   (1 B)          number of levels
bits       (n_levels B)   bit cleared at each level
flags_len  (4 B BE)
flags_gz   (flags_len B) gzip(concat of all n_levels flag bitsets)
term_len   (4 B BE)
term_gz    (term_len B)   gzip(final aligned stream)
```

CAIM does not participate in the HALT_ANS_STRIDE optimisation. The HALT_ANS_STRIDE halt condition is specific to the RECURSIVE mode.

7.4 HALT_ANS_STRIDE Reconstruction at Decode

When the decoder encounters `halt = 4` on a level record, it decodes the ANS-stride payload to recover the aligned stream for that level. The reconstruction proceeds as follows:

- Decode ANS-stride payload → get `aligned` (the bit-cleared stream, pre-remap)
- Decode `flag_data` using the level's `fmt` → get flag positions `F`
- Call `reconstruct(aligned, F, bit)` → restore the original stream at this depth
- Do **not** call `unremap()` for this level: `aligned` was never remapped during encode

- Continue the bottom-up reconstruction loop for all shallower levels normally (unremap + reconstruct)

8. Core Operations Reference

8.1 remap(A, b*)

```
for each byte v in A:
    high = v >> (b* + 1)
    low  = v & ((1 << b*) - 1)
    v'   = (high << b*) | low
```

8.2 unremap(R, b*)

```
for each byte v in R:
    high = v >> b*
    low  = v & ((1 << b*) - 1)
    v'   = (high << (b* + 1)) | low
```

8.3 reconstruct(A, F, b*)

```
result = bytearray(A)
for p in F:
    result[p] |= (1 << b*)
return bytes(result)
```

8.4 sweep(data, max_bit)

```
counts = [0] * 8
for byte in data:
    for b in range(max_bit + 1):
        if (byte >> b) & 1: counts[b] += 1
return argmin(counts[:max_bit + 1])   # lowest index on tie
```

8.5 bit_clear(data, b*)

```
mask    = ~(1 << b*) & 0xFF
aligned = bytes(v & mask for v in data)
flags   = [i for i, v in enumerate(data) if (v >> b*) & 1]
return aligned, flags
```

9. Complete Encode Algorithm (Recursive Mode)

```
def recursive_encode(data):
    n = len(data)
    levels = []   # list of (bit, fmt, flag_block, ans_candidate)
    current = data

    # ---- Descent ----
    for depth in range(MAX_DEPTH):
        max_bit = 7 - depth
        if max_bit < 0: break
        if is_all_zero(current) or is_all_one(current): break

        best_bit         = sweep(current, max_bit)
        aligned, flag_pos = bit_clear(current, best_bit)
        ans_candidate    = ans_stride_encode(aligned)   # <-- v15 addition
        remapped         = remap(aligned, best_bit)

        density    = len(flag_pos) / len(current)
        flag_block = flag_race(flag_pos, len(current), density)

        levels.append((best_bit, best_fmt, flag_block, ans_candidate))
        current = remapped

    # ---- Terminal ----
    if is_all_zero(current):
        inner = pack_header(0, FMT_GAP, 0, HALT_ZERO, 0)
    elif is_all_one(current):
        inner = pack_header(0, FMT_GAP, 0, HALT_ONE,  0)
    else:
        gz    = gzip_compress(current)
        inner = pack_header(0, FMT_GAP, 0, HALT_TERMINAL, len(gz)) + gz

    # ---- Bottom-up assembly with optimal cutoff ----
    payload = inner
    for (best_bit, best_fmt, flag_block, ans_cand) in reversed(levels):
        ans_cost     = 11 + len(ans_cand)          # header + payload
        recurse_cost = len(payload)                # everything below

        if ans_cost < recurse_cost:
            # HALT_ANS_STRIDE wins at this depth
            hdr     = pack_header(best_bit, best_fmt, len(flag_block),
                                  HALT_ANS_STRIDE, len(ans_cand))
            payload = hdr + flag_block + ans_cand
        else:
            hdr     = pack_header(best_bit, best_fmt, len(flag_block),
                                  HALT_RECURSE, len(payload))
            payload = hdr + flag_block + payload

    return payload
```

10. Complete Decode Algorithm (Recursive Mode)

```
def recursive_decode(payload, n):
    levels = []         # list of (bit, fmt, flag_data)
    ans_terminal = False
    pos = 0

    # ---- Parse level headers ----
    while True:
        bit, fmt, flag_len, halt, child_len = parse_header(payload, pos)
        flag_data = payload[pos+11 : pos+11+flag_len]
        pos      += 11 + flag_len

        if halt == HALT_RECURSE:
            levels.append((bit, fmt, flag_data))
            # continue to child
        else:
            child = payload[pos : pos + child_len]
            if   halt == HALT_ZERO:       current = bytes(n)
            elif halt == HALT_ONE:         current = bytes([1] * n)
            elif halt == HALT_ANS_STRIDE:  current = ans_stride_decode(child)
                                           ans_terminal = True
            else:                          current = gzip_decompress(child)
            if flag_len > 0:
                levels.append((bit, fmt, flag_data))
            break

    # ---- Reconstruct bottom-up ----
    for i, (bit, fmt, flag_data) in enumerate(reversed(levels)):
        is_ans_level = ans_terminal and (i == 0)
        if is_ans_level:
            aligned = current        # already aligned, no unremap needed
        else:
            aligned = unremap(current, bit)
        flag_pos = decode_fmt(flag_data, n, fmt)
        current  = reconstruct(aligned, flag_pos, bit)

    return current
```

11. Integrity Verification

The container header includes a SHA-256 digest of the original uncompressed data. Decoders must verify this before returning data:

```
expected = header.sha256
actual   = sha256(decoded_data)
if actual != expected:
    raise IntegrityError('SHA-256 mismatch')
```

Verification may be disabled for performance in trusted contexts, but must be the default behaviour.

12. Reference Results

12.1 v15 vs v14 vs gzip (5 MB test files, aim_v15.py)

File	Raw Size	gzip lvl 9	AIM v14	AIM v15	v15 vs gzip	v15 vs v14
PCM Audio	5,000,000	4,697,141	4,418,798	3,805,723	−19.0%	−13.9%
Synthetic YUV420	5,000,000	4,647,141	3,447,235	2,856,589	−38.5%	−17.1%

12.2 Full WAV Benchmark (38.7 MB, aim_v14.py reference)

File	Size	gzip ratio	AIM ratio	vs gzip	Mode
sample-full.wav	38.7 MiB	100.00%	84.97%	−15.03%	Recursive

The −15.03% result on the 38.7 MB WAV file was confirmed analytically before the C implementation ran and verified exactly on first execution in Session 21. This benchmark was computed with aim_v14.py. The v15 result on the same file is expected to be larger gains; this benchmark will be updated when the v15 C implementation is completed.

12.3 Correctness Invariants

Any conforming implementation must satisfy:

- SHA-256 of decoded output equals SHA-256 of original input for all inputs.
- Encoding the output of the decoder reproduces a valid container (not necessarily byte-identical: gzip implementations vary, but all valid containers decode to the same original data).
- HALT_ANS_STRIDE never fires on random data (gain below threshold).
- All seven flag formats produce the same decoded flag set for any given flag_data/n/fmt triple.

13. Implementation Notes

13.1 Memory

Both the recursive encoder and decoder are iterative, not recursive. Peak memory is O(n): only one copy of the working data is live at any time. ANS-stride candidates are held as byte buffers during descent and discarded after assembly. For MAX_DEPTH = 8, at most 8 candidate buffers exist simultaneously, each at most n bytes; in practice they are much smaller.

13.2 Threading

The flag race evaluates up to 7 formats independently and can be dispatched to a thread pool. ANS-stride candidate computation is independent of the flag race and can also run in parallel. For `n >= PARALLEL_MIN (512 KB)`, dispatch flag race and ANS candidate to the thread pool simultaneously.

13.3 C Implementation Notes

The algorithm maps cleanly to C. The iterative encode and decode structures are explicit stack loops over fixed-size frame arrays. The sweep is a tight inner loop. The rANS encode loop is a sequential operation (each symbol depends on the previous state) but is branchless-friendly and cache-local. The frequency table normalisation and slot array construction can be SIMD-vectorised. The reference C implementation (`aim.c`) implements v14; the v15 architecture (HALT_ANS_STRIDE) has not yet been ported.

13.4 Edge Cases

Input	Expected Behaviour
Empty (n=0)	Valid container. HALT_TERMINAL with gzip(empty).
All-zero bytes	HALT_ZERO fires at depth 0. child_len = 0.
All-0xFF bytes	Any bit has density 1.0. HALT_ONE fires at depth 0.
Already compressed	All ANS-stride gains below threshold. gzip wins terminal. Output near raw size.
Random data	All formats compete. LZ77HUFF wins at ~50% flag density. ANS-stride does not activate.

AIM Canonical Specification v16 — Reference implementation: aim_v15.py — github.com/Jacob-Ritchey/AIM — MIT License